User Manual

CoreBus (version 3.5.2)

Support the project

andrey.abdulkayumov@gmail.com

Table of contents

Brief description

CoreBus (formerly known as “Terminal Program”) is a cross-platform terminal for working with serial ports and TCP sockets with support for Modbus TCP / RTU / ASCII protocols.

Main features of the application:

1. Three operating modes: “No protocol”, “Modbus” and “Modbus monitoring”.

2. "No protocol":
   - Working with data in string or byte format.
   - Support for different encodings.
   - Three sending modes: single, cyclic, file sending.

3. "Modbus":
   - Supports various variations of the Modbus protocol: TCP, RTU, ASCII and RTU / ASCII over TCP.
   - Convenient operation with recording functions.
   - Ability to work with float numbers.
   - Ability to work with binary data.
   - Modbus scanner that searches for devices on the communication line.

4. "Modbus monitoring":
   - Convenient display of registers.
   - Conversion to numeric types (Int16/32, float, etc.).
   - Conversions according to a given formula.
   - Plotting a chart in real time.
   - Logger.

5. Macros:
   - Separate macros for each operating mode.
   - A macro consists of an unlimited number of commands (actions).
   - For Modbus macros, it is possible to set a common Slave ID for the entire macro.
   - Import and export of macros.

6. Dark and light application themes.

7. Presets with custom settings.

8. User manual (Russian, English).

9. Multi-language interface support: Russian, English, Hindi, Chinese (Simplified), German, Spanish, French, Korean, Portuguese.

10. Cross-platform: Windows, Linux.

The application was tested on Windows 10/11, Linux Ubuntu.

A demonstration of the application can be viewed by following the link.

No protocol

In the transfer field, the user writes the data that needs to be sent. The receive field contains the data sent by the server or external device. You can work with both bytes and string data in different encodings.

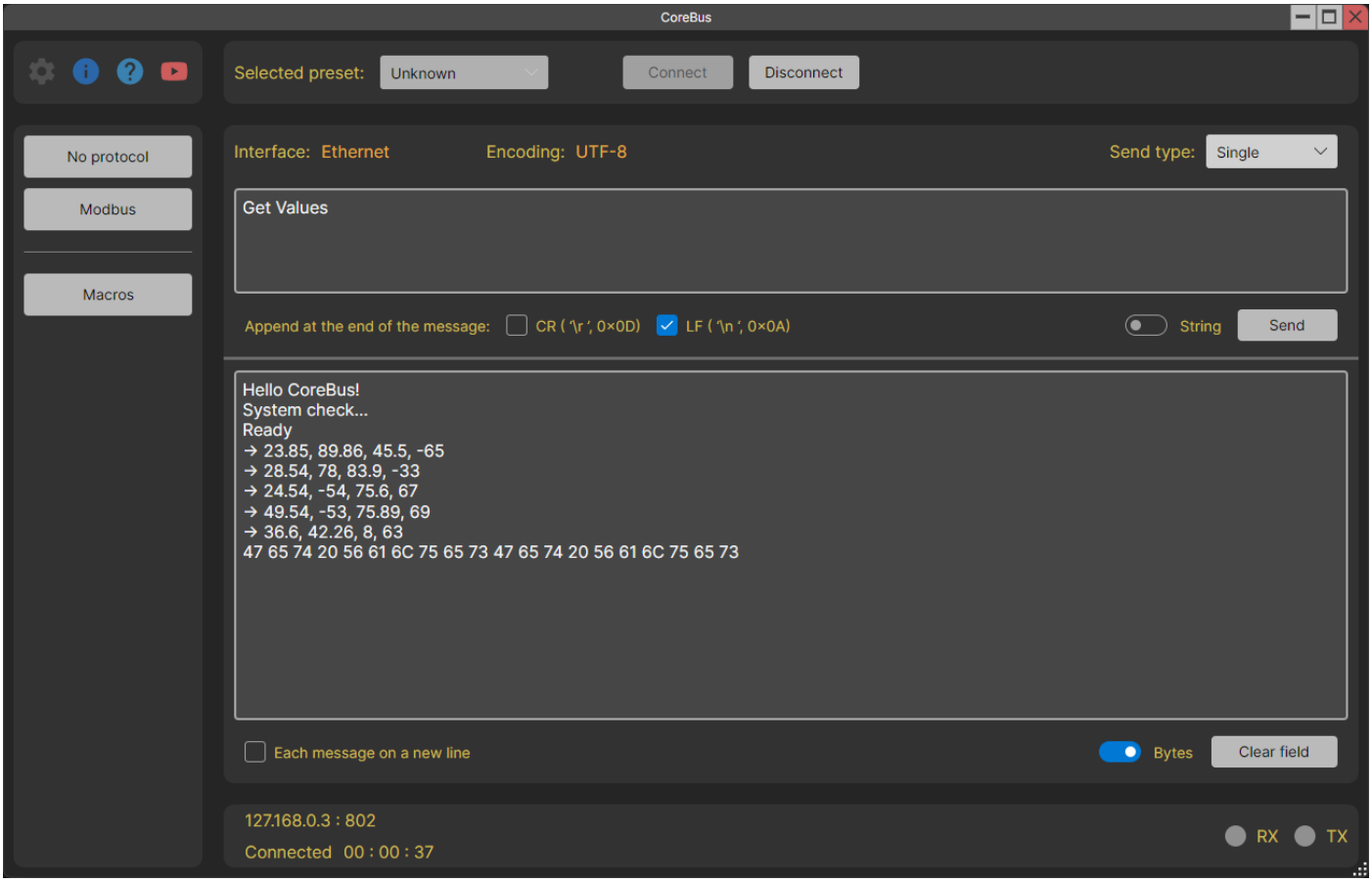
Supported protocols:

- UART
- TCP

There are three sending modes:

- Single
- Cyclic
- Files

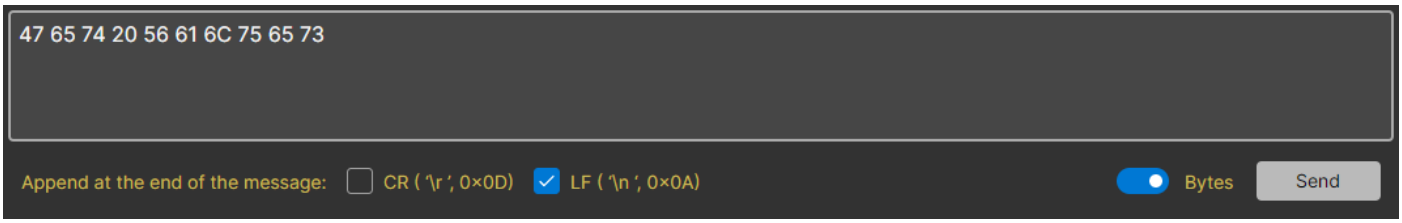
The sending type is selected in the drop-down list in the upper right corner.



Single send

In this mode, you can send bytes or a string to the connected host. Sending occurs once by clicking on the “Send” button. You can also add special characters at the end of the message.

The string encoding is set in the mode settings.

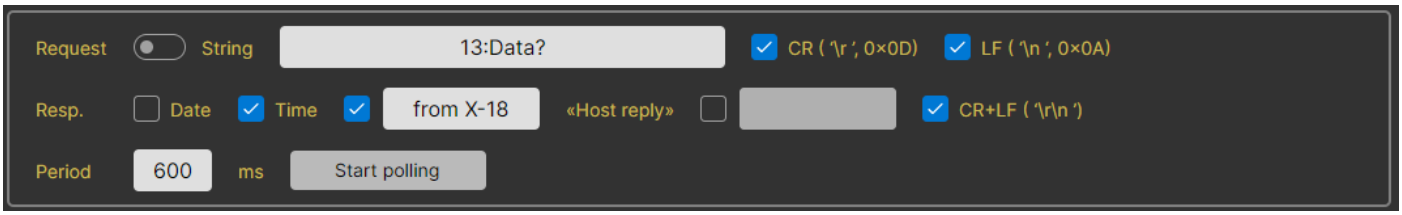


Cyclic sending

This mode allows you to send a message to the host automatically at a specified time interval.

The functionality of the “Request” line is similar to the normal mode of operation. And in the “Response” line, you can add service information to the message itself: the date the message was received in the format DD.MM.YYYY, time in the format HH:MM:SS, custom lines at the beginning and/or end of the message and service characters.

The string encoding is the same as in normal mode.

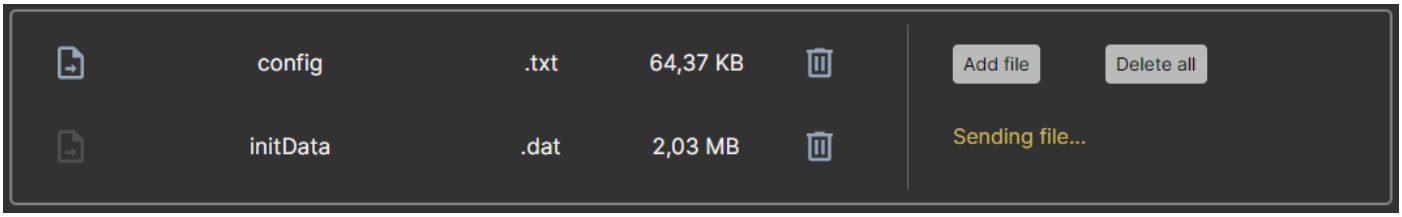


Sending files

This mode allows you to send files. The specified encoding is not used in this mode.

Files can be reused multiple times. They are added to a special folder, so they do not need to be added every time the application is launched.

While the file is being sent, the message “Sending file...” appears. Then she disappears.



Important note!

Sometimes it may seem that the application is not accepting data in “No Protocol” mode. The receive field is empty or contains garbage, although the host sent meaningful data.

Explanation:

This may be because the Receive field's String/Bytes selector is set to display a string, and the host is sending a set of bytes that could not be converted to a string.

If string display is selected in this selector, the application attempts to convert all received bytes into a string of the specified encoding.

Solution:

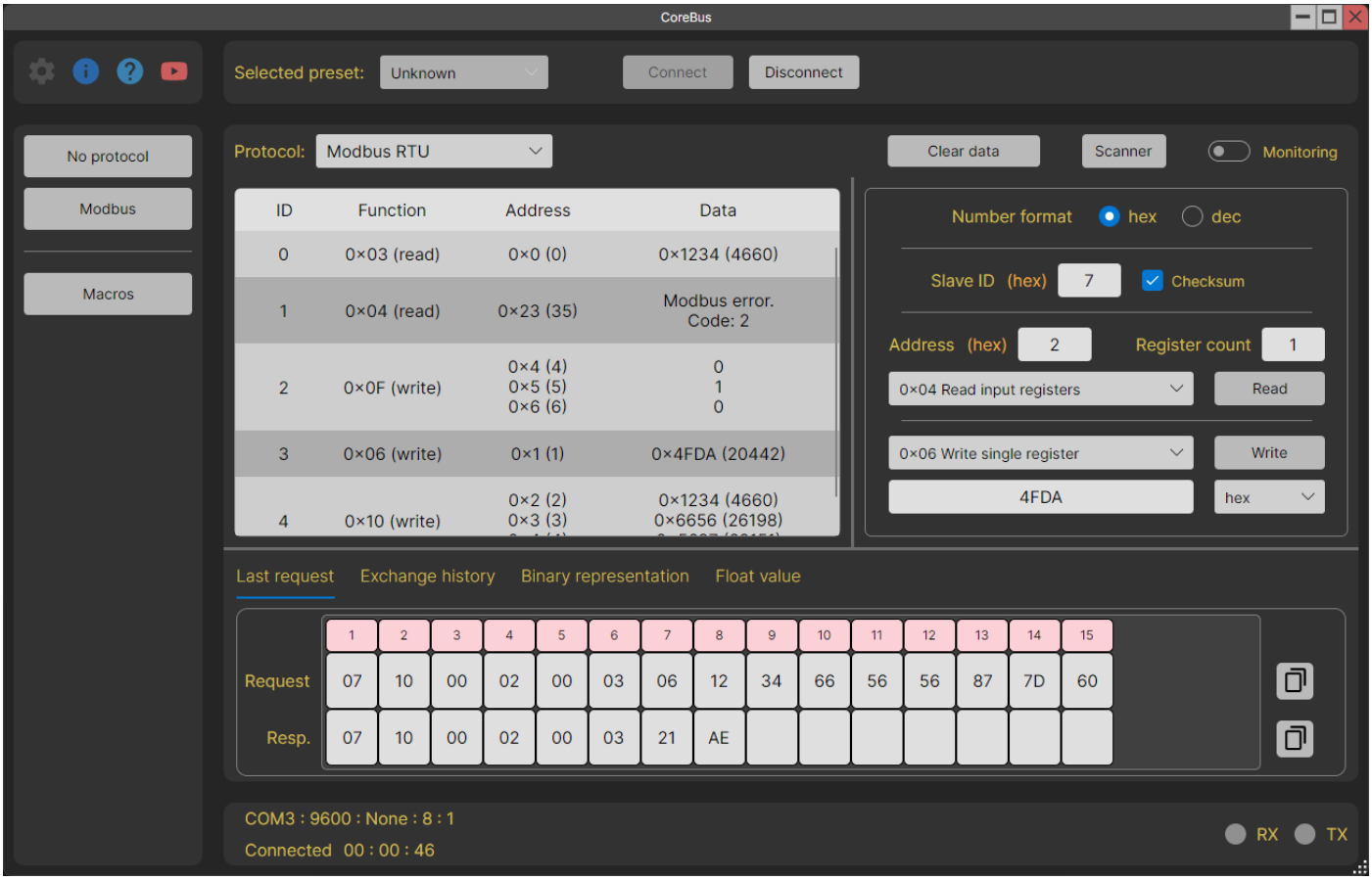
Set the “String / Bytes” selector in the receive field to the “Bytes” state.

Modbus

The user can interact with selected Modbus registers using the appropriate interface elements. For further decryption of the transaction, there is a section with views.

Supported protocols:

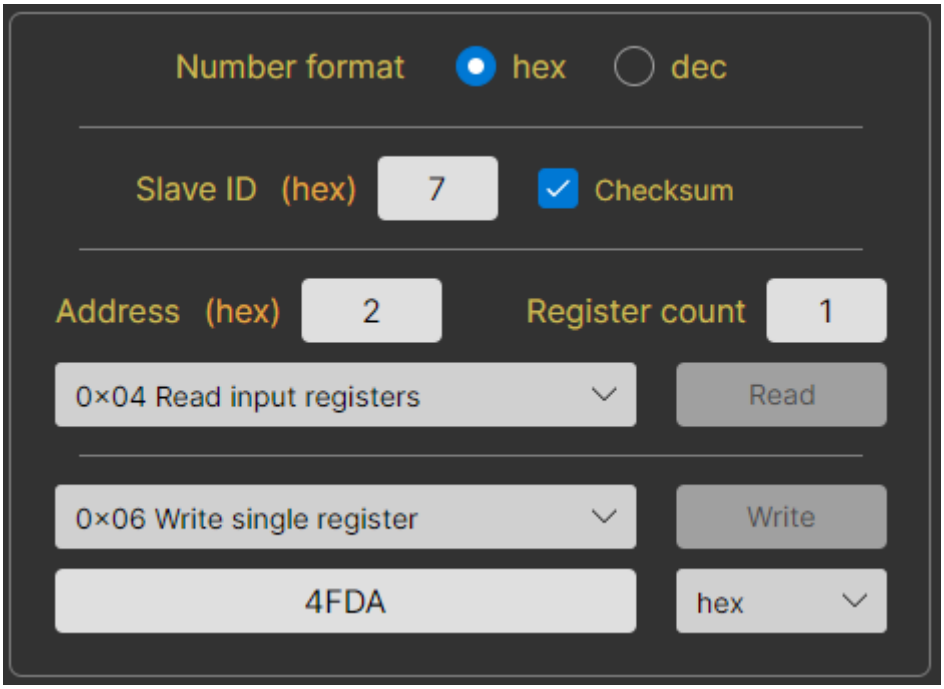
- Modbus TCP
- Modbus RTU
- Modbus ASCII
- Modbus RTU over TCP
- Modbus ASCII over TCP



Normal operation

During normal operation, Modbus registers can be read or written.

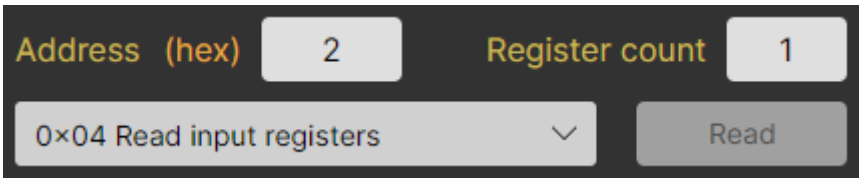
The “Number Format” switches change the number format in the “Slave ID” and “Address” fields. The selected format is indicated in brackets next to these fields.



This mode is the richest in functionality, so let’s look at it in more detail.

Reading Modbus registers

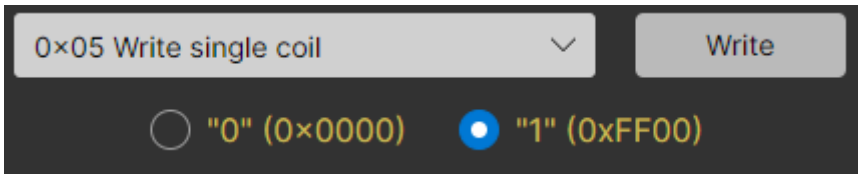
Select the function, start address, register count and press the “Read” button.



Writing Modbus Registers

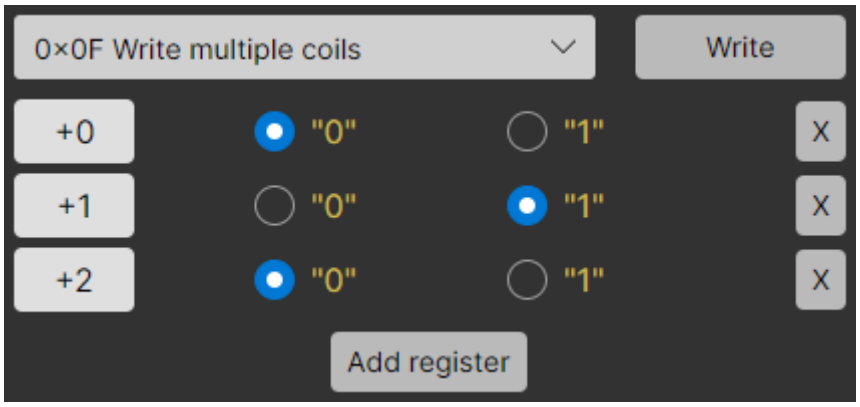
Each function has its own design option. The starting address for all functions is the value from the "Address" field.

0x05 Write one flag



According to the protocol documentation, the data field must contain only one of two values. 0x0000 is a logical zero, and 0xFF00 is a logical one. Therefore, select the desired value and press the “Write” button.

0x0F Write multiple flags

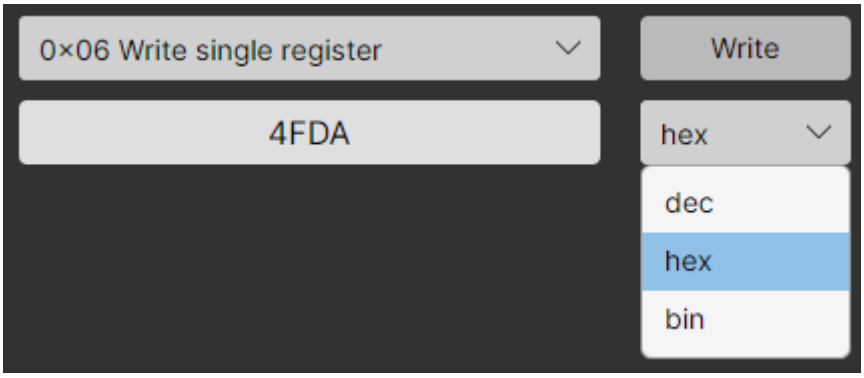


Using the “Add register” button, we create the required number of flags, set the value and click the “Write” button.

To the left of the register values, we have offset values relative to the starting address.

On the right are delete buttons for each register.

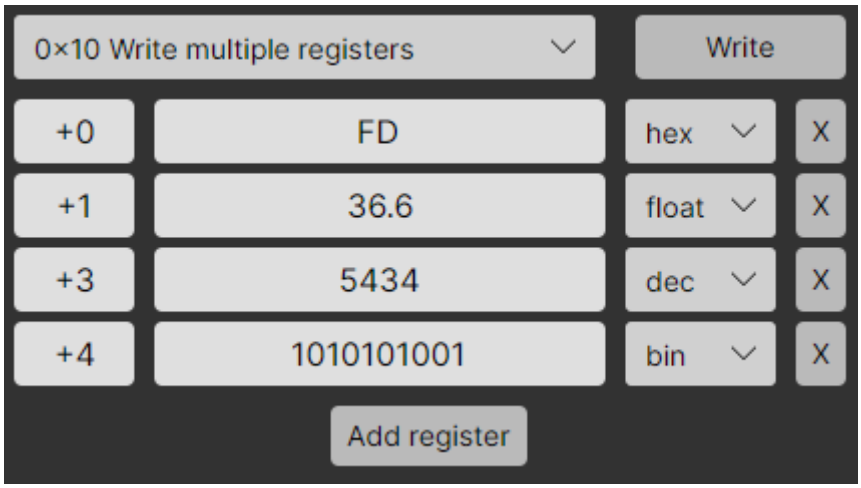
0x06 Write one register



Using this function, you can write to 16-bit registers.

The format of the number to be written is selected in the drop-down list to the right of the input field. When changing the format, the number is automatically converted.

0x10 Write multiple registers

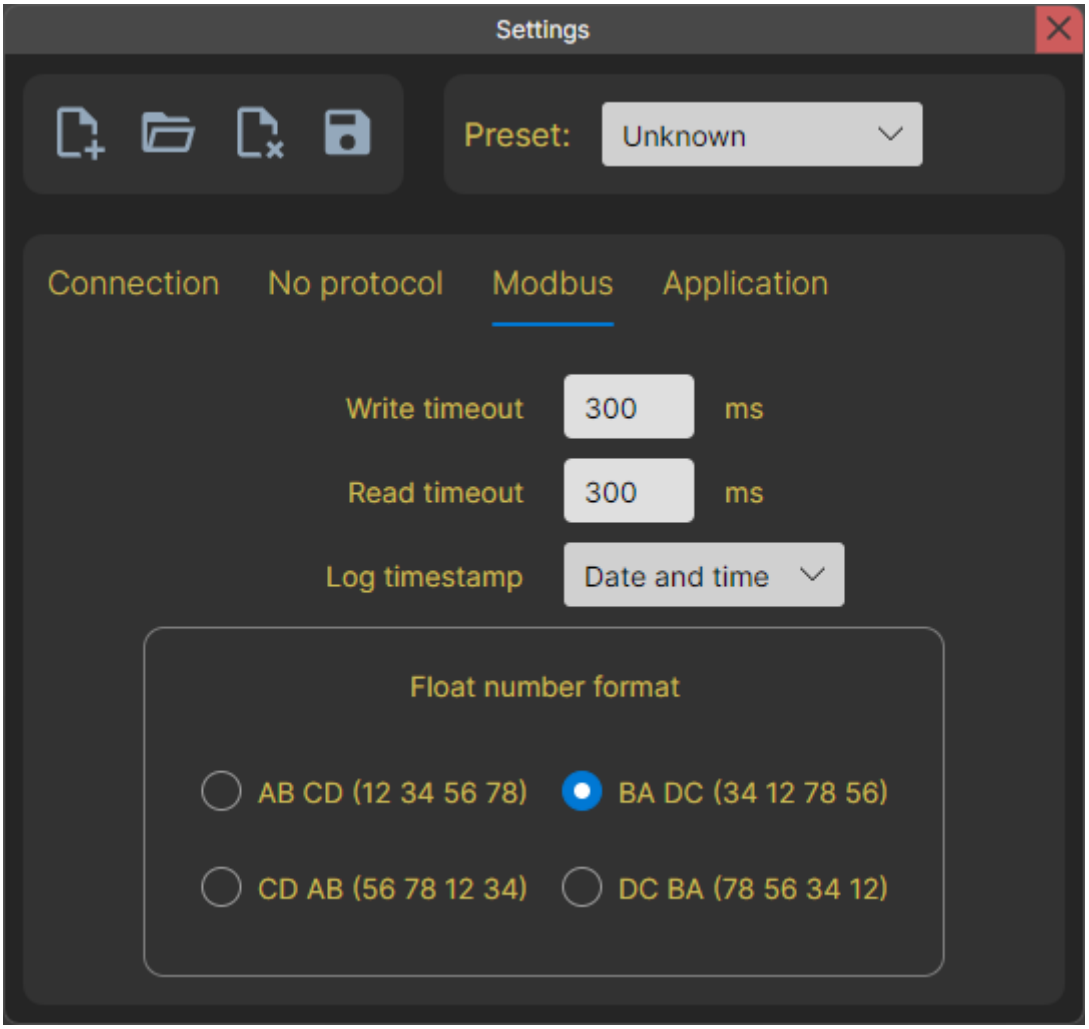


The control here is similar to the “0x0F Write multiple flags” function.

This function makes it possible to write numbers of the float type.

Such numbers occupy 2 words or 4 bytes. Therefore, the next register has an offset of not +1, but +2 addresses.

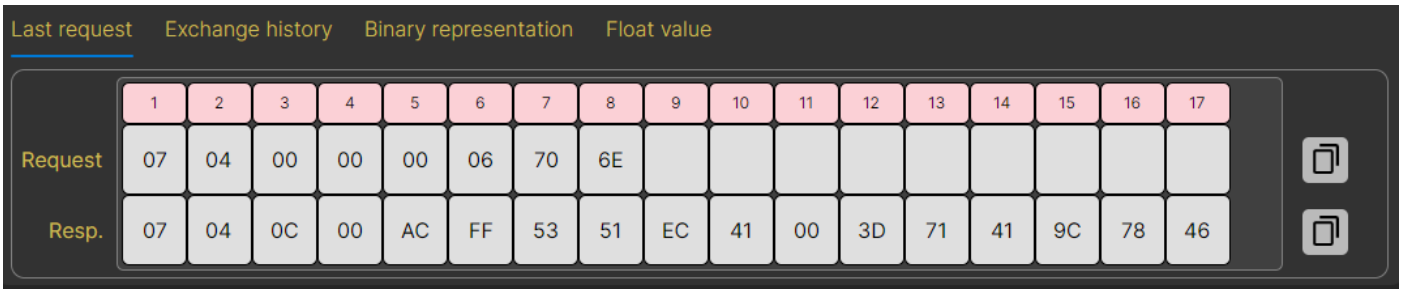
Sometimes it happens that a device may use an atypical format to decrypt float numbers. And to adapt to a specific device, you can select the desired recording format in the settings.



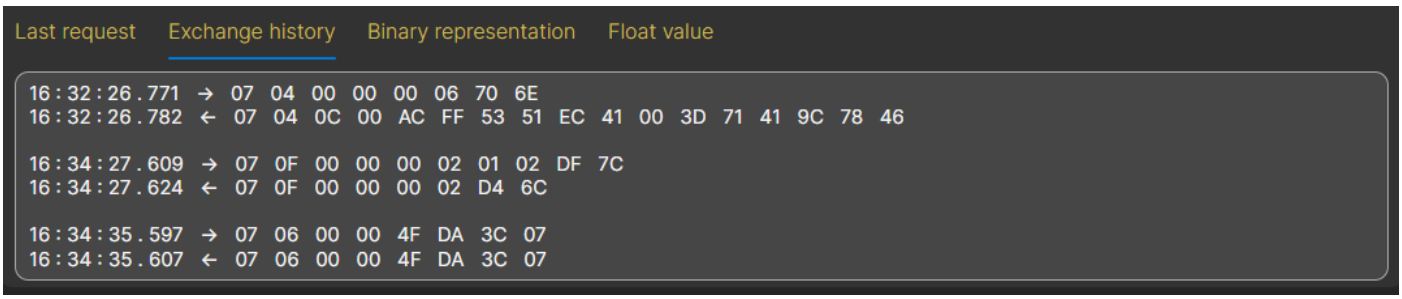
Modbus mode settings page.

You can simply view the register values in a table view. But unfortunately, these numbers do not always make sense. And sometimes they need to be “deciphered”. Therefore, for interpreting data, the terminal provides an area with views.

There are 4 types of views in total:

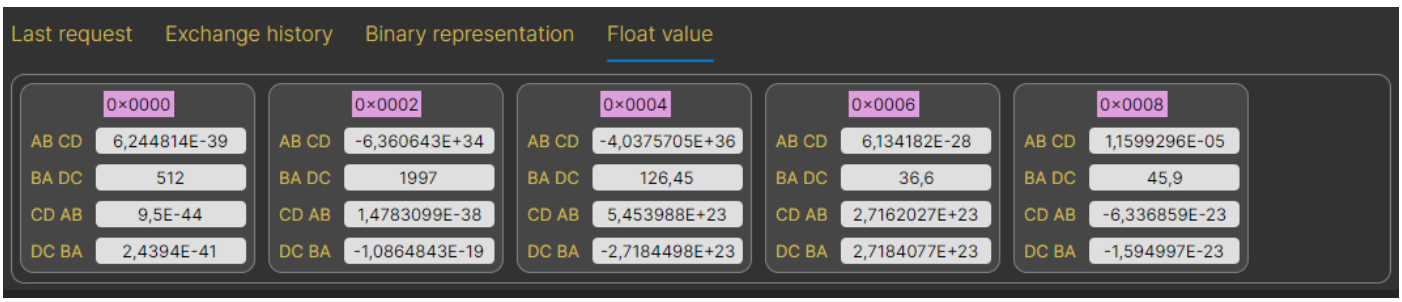


Last request



Exchange history

Binary representation



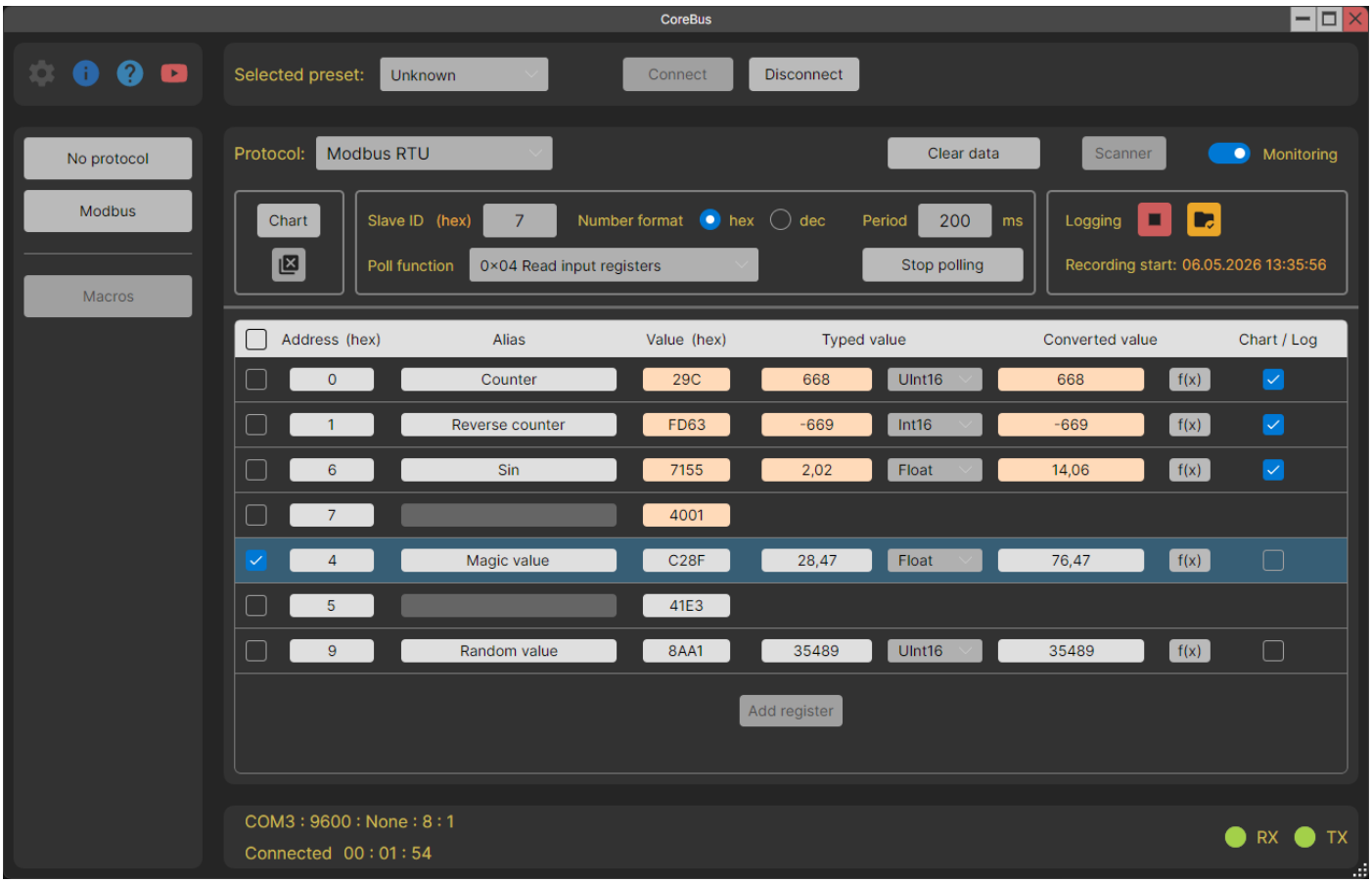
Representation of a float number

Modbus monitoring

A special mode designed for visual control of the connected device.

In this mode, Modbus registers are displayed, the values of which are updated with a specified period. For convenient display, these values can be converted to the selected type, converted using a formula and displayed on a graph.

The transition to it occurs by pressing the “Monitoring” switch in Modbus mode.



Globally, the working field is divided into two parts. The upper part is responsible for managing monitoring. And at the bottom there is a list of polled registers.

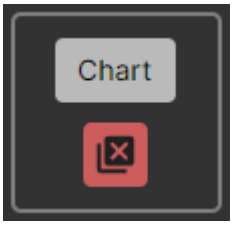
Important!

Registers are polled with a specified period. If the device fails to respond during the current polling cycle, the next polling cycle is skipped until a response is received or an error occurs. Thus, a situation is possible when the period is set to 10 ms, and the actual data update occurs at intervals of 150 - 200 ms.

Monitoring control field

It consists of three groups of elements. Let's look at them from left to right.

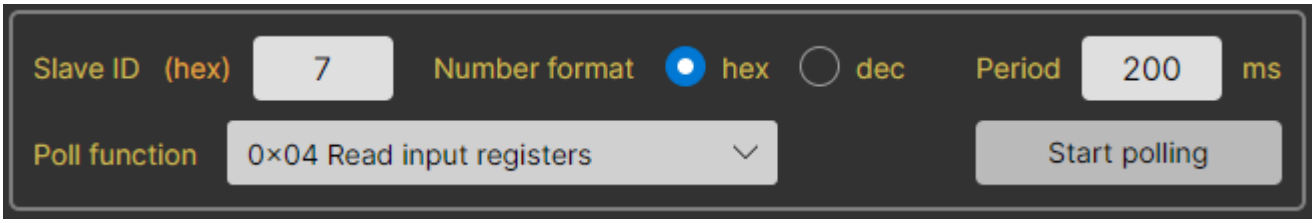
UI element management group.



At the top is a button that opens the chart.

Below is a button that deletes selected registers.

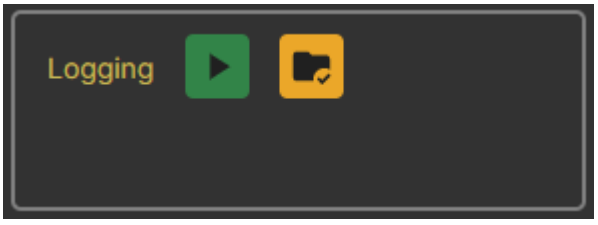
Monitoring settings group.



Here you can select the required polling parameters. As well as the number format that will be applied to the “Slave ID” field and the “Address” and “Value” fields in the list of registers.

A logging group that has two states.

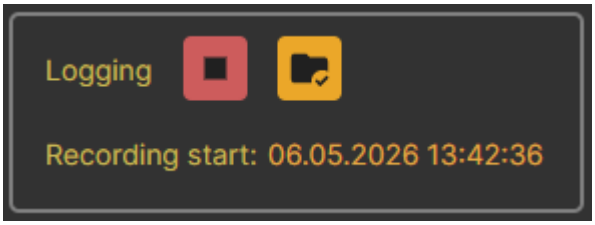
Inactive.



There are only two buttons here.

On the left is a button to start logging, and on the right is a button to open a folder with logs.

Active.



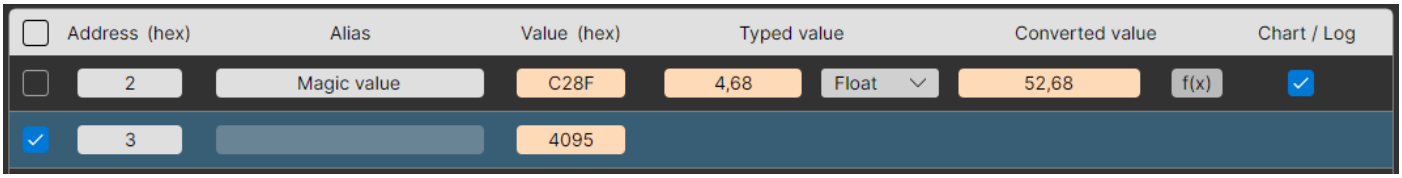
The start button turns into a stop logging button.

And the date and time the recording started appears at the bottom.

List form of registers

Each line is one 16-bit Modbus register.

Let's look at the list elements using this example.



The checkmarks on the left mark the case for deletion or simply for visual highlighting among other elements of the list. The register with address 3 is highlighted in the picture.

1. **Address**
   Contains the address of the register. May be in HEX or DEC format.

2. **Alias**
   An arbitrary symbolic name. If it is empty, then on the graph and in the log the register will have the name “Address + address value”.
   For example, “Address 3”, “Address 75”, etc.

3. **Value**
   "Raw" register value. May be in HEX or DEC format.

4. **Typed value**
   A register value cast to one of the available types. If the type takes up more than 16 bits, then the following values are also captured during conversion (as in the picture).

   For example, the float type takes up 32 bits, which means that two Modbus registers are required to convert a value to this type.

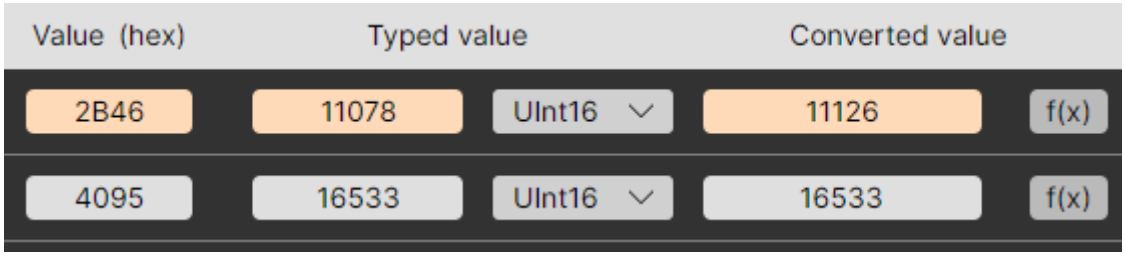
5. **Converted value**
   A typed value converted using a given formula. The formula is specified in a window that opens by clicking on the “f(x)” button. See below for more details.

   In the example in the picture the formula is “0.5*x”.

6. **Chart / Log**
   This selection means that the converted value of this register will be displayed on the chart and written to the log.

Values in the “Value”, “Typed value”, and “Converted value” columns are highlighted if they have changed since the last polling iteration.

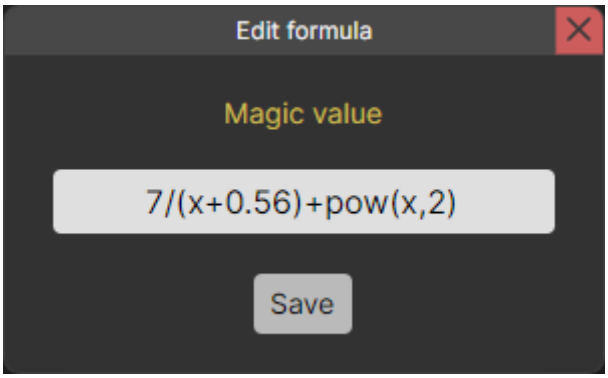
The values above have been changed and are therefore “highlighted”.



Conversion by formula

A custom formula converts the value from the “Typed value” column and writes it to the “Converted value” column. By default, the formula is "x", i.e. the field contains a copy of the value from the “Typed value” column.

Formula editing is available in the inactive state. And during a polling, only viewing is allowed.



You can also view the formula by hovering the cursor over the “f(x)” button.

Important!

Trigonometric functions take an argument and return a result in radians.

Logging

The application supports logging function. You can start and stop it at any time using the buttons in the monitoring control field (see above).

Only those registers that have the “Chart / Log” checkbox checked in the list of registers are written to the log. The values are taken from the “Converted value” column and stored in full form - without rounding, as a double number.

The log file is saved in .txt format. You can add a timestamp to each entry. Its format is configured in the “Settings” → “Modbus” → “Log timestamp” section.

Several label formats are available:

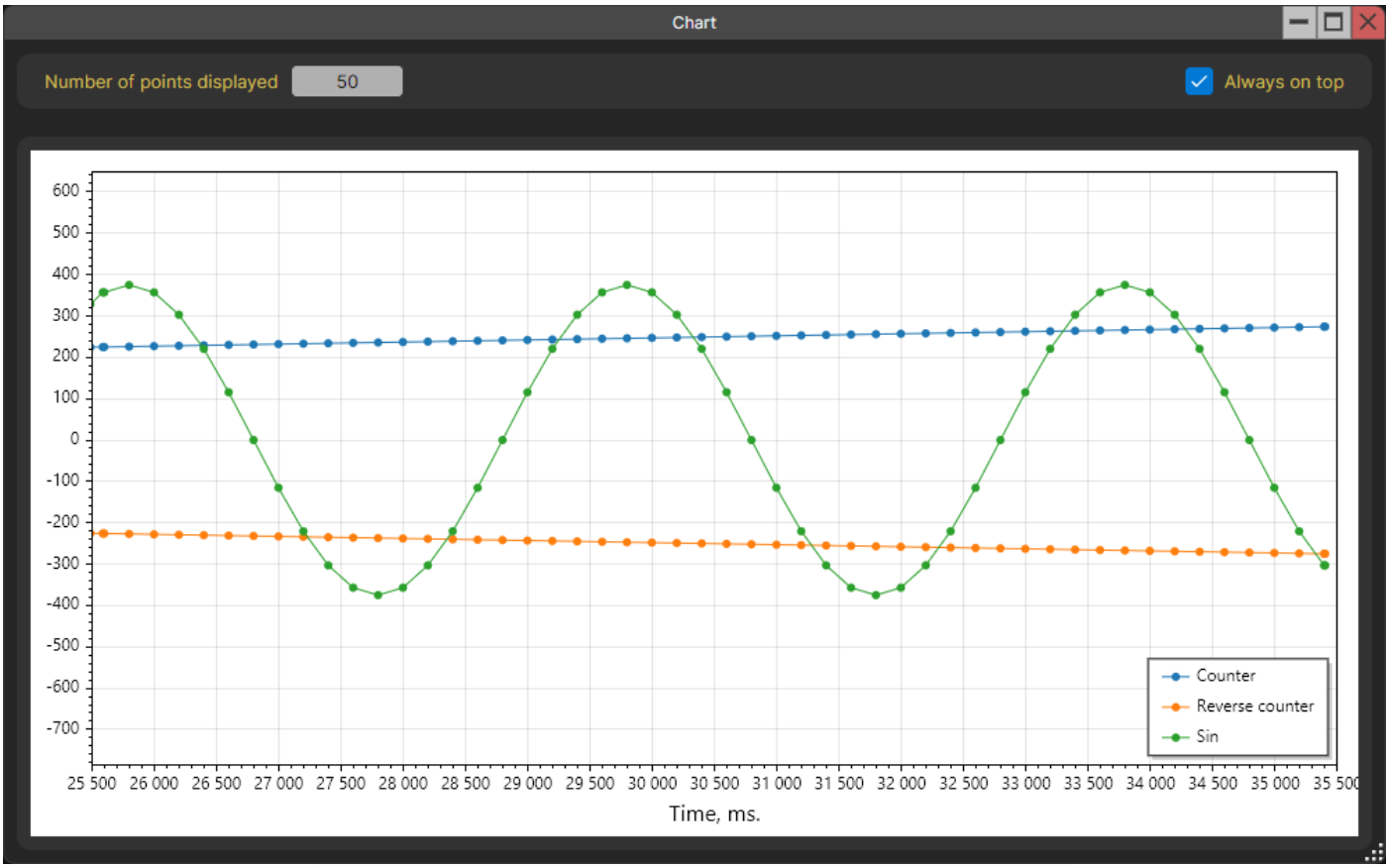
- None
- Time only
- Date and time
- ISO 8601

The default format is “Date and Time”.

Chart

The application can build a chart in real time.

The chart displays only those registers that are marked with the “Chart / Log” checkbox in the register list form. The values are taken from the “Converted value” column.



Only two settings are available to the user:

- Select the number of points to be displayed.
- Placement of the chart window on top of all windows.

Both settings are remembered after closing the application.

Modbus scanner

The Modbus scanner is used to search for slave devices on the communication line. This function is only available when connected via a serial port, because when connecting via TCP/IP, it makes no sense.

The search can take place via the Modbus RTU or Modbus ASCII protocol. The protocol type is selected in the drop-down list.

The “Devices” field will display the addresses of the found devices. If after the end of the search this field is empty, this means that no device responded in the entire range of valid addresses (1 - 255). Broadcast address 0 is not taken into account because according to the documentation, devices should not respond to it.

The top right field indicates the PDU that is being queried.

It is important to understand how the pause between sending messages works. This pause consists of two components - a user timeout, which is indicated in the “Pause” field, and a read timeout, during which the application waits for a response from the device. The reading timeout is set in the settings.

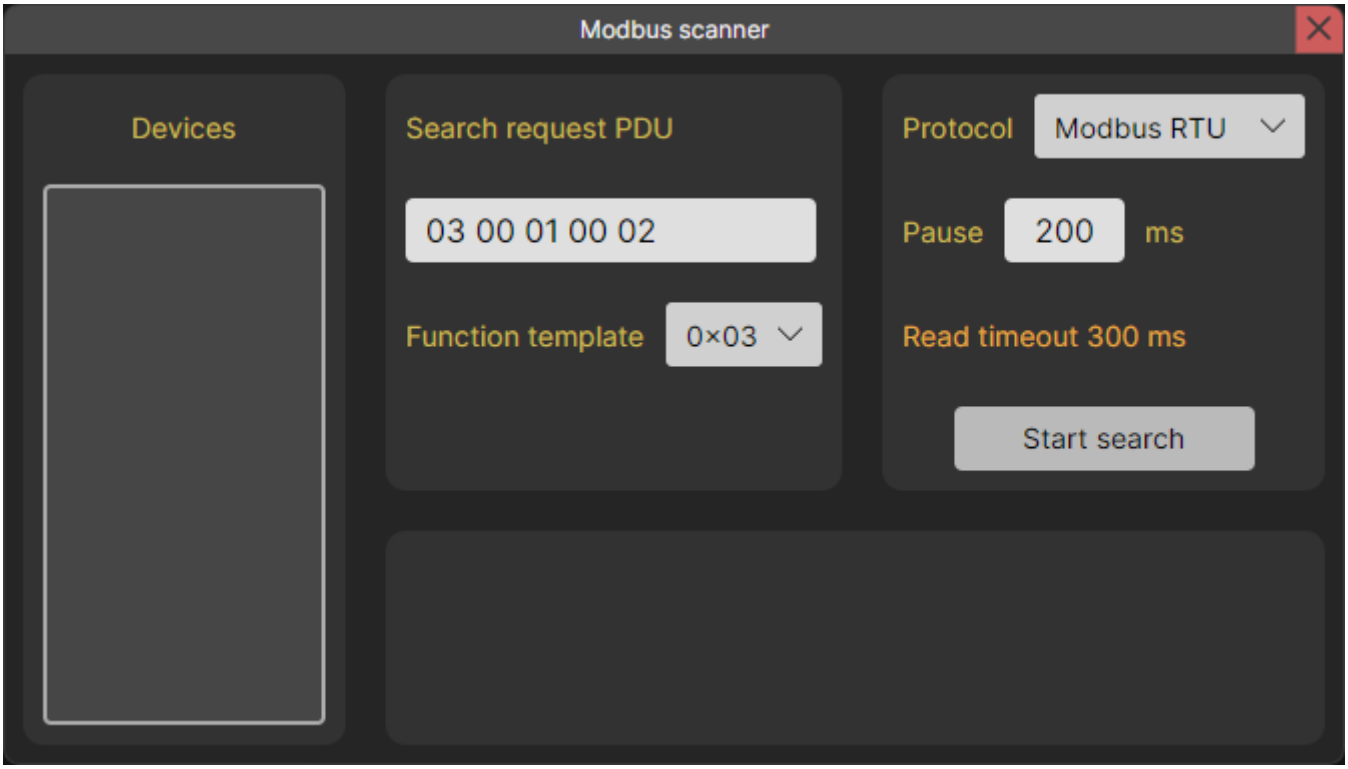
Important!!!

It is possible that after polling all addresses, the “Devices” field is empty. At the same time, it is reliably known that there are properly functioning devices on the communication line.

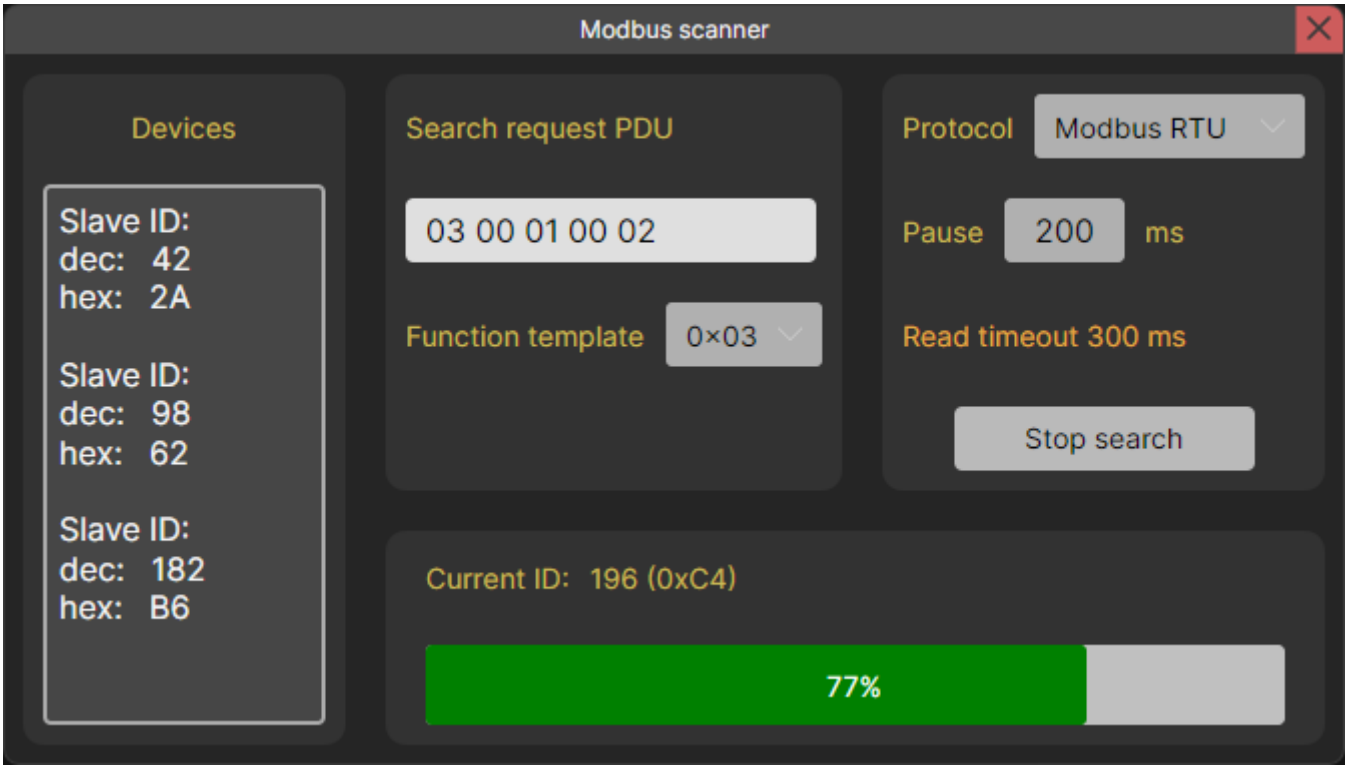
This could be due to two reasons:

1. For some reason, the slave device did not have time to process the message and send a response. In such cases, it is recommended to increase the user timeout in the “Pause” field.

2. The device you are looking for does not respond to the specified polling function. You can try to search for it by selecting any other function in the “Function template” drop-down list.



Inactive state



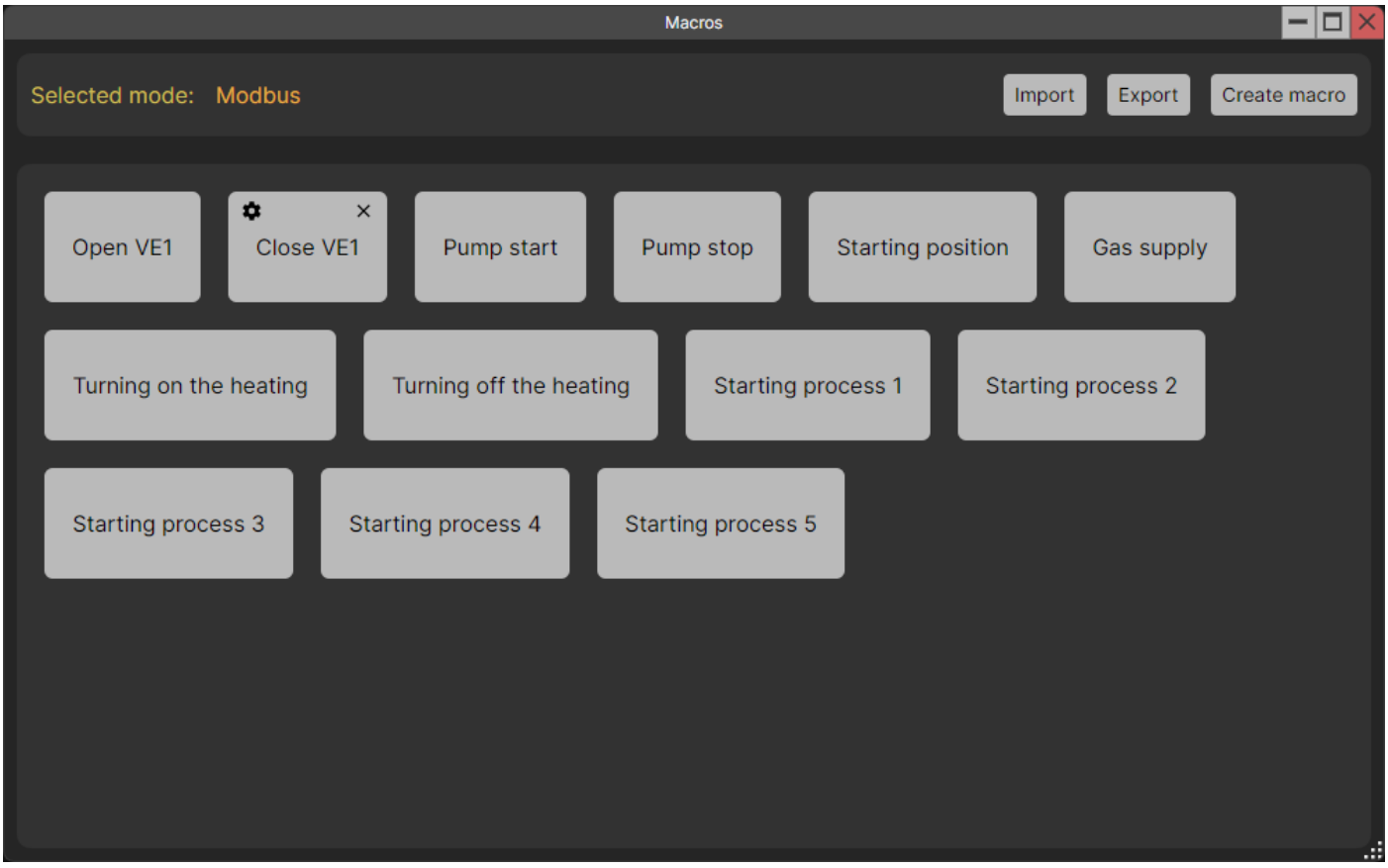
Search state

Macros

The application allows for working with macros. They are available for all modes. Macros support sending multiple messages at once.

All macros are presented on the workspace in the form of buttons with appropriate names. When you hover over any of the macros, edit and delete buttons appear.

It is also possible to import and export a macro file for each mode. This is convenient to use when you need to transfer macros to several PCs.



Editing a Macro

The macro is divided into commands. Each command is sending one message.

When you open the macro editing window, the original window with the work field is hidden. And after closing the editing window, it appears again. This is made for more independent work with windows.

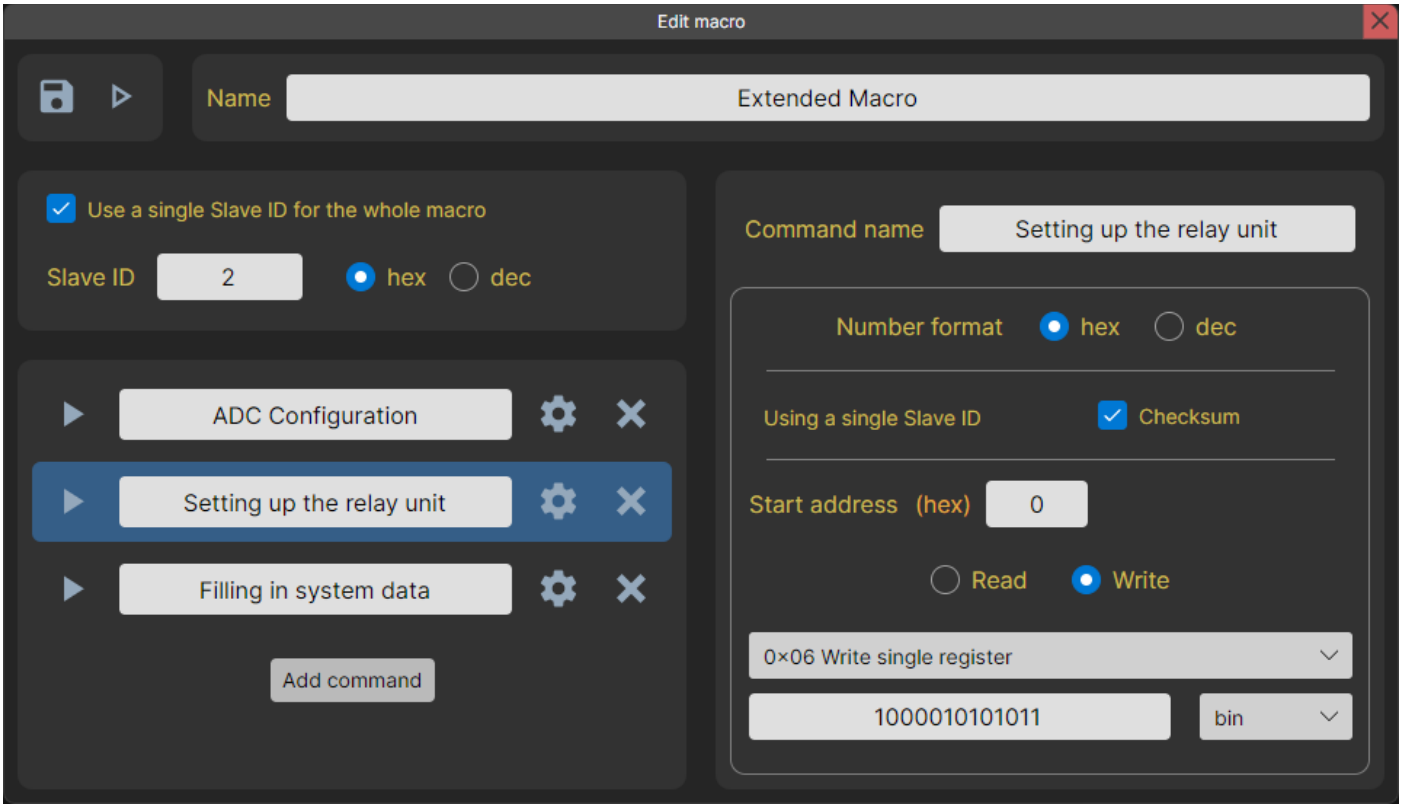
In editing mode, it is possible to send individual commands or the entire macro. For this purpose, there are corresponding buttons in the macro header and for each command in the list form.

The working field is divided into four parts.

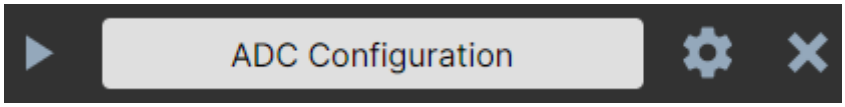
At the top there is a field for editing the name of the macro, and to the left of it there are buttons for saving and running the macro. Before saving or launching, a check occurs. If there are errors, a message appears listing all the errors found.

The lower part is divided in half. On the left there is a list with all the macro commands and a button to add a command, and on the right there is a form for editing the selected command. The command being edited is highlighted in the list.

For the “Modbus” mode, it is possible to set a common Slave ID for the entire macro.



Let's look at the elements of each command from the list.



From left to right.

- **Command launch button.**
  If the command contains no errors and the host is connected, the message will be sent. Otherwise, a message will appear describing the error.

- **Field with the command name.**
  It is only available for selection and copying. The command name can only be changed in the editing form.

- **Button to open/close the command editing form.**
  The command being edited is highlighted in the list. You can also, without closing the edit form for the current team, click on the same button for another team and edit it.

- **Delete command button.**
  You can’t just delete it; you need to confirm the deletion in the dialog box.

Each operating mode has its own command editing form. Let's look at them separately.

"No protocol" mode command

The interaction in this window is similar to the usual “No protocol” mode.

Important!

The encoding of a string in a macro is autonomous.

It does not depend on the general encoding specified in the application settings.

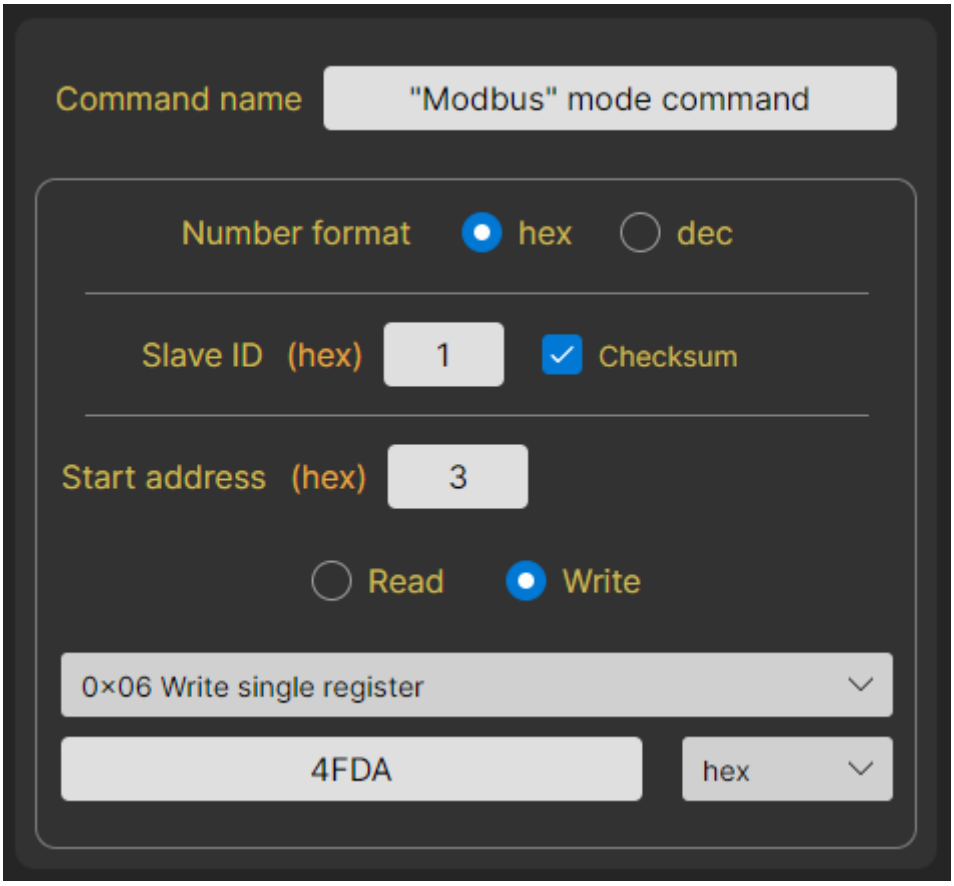
Modbus mode command

The control in this window is also similar to the usual “Modbus” mode.

Important!

The format of a float number in a macro is self-contained.

It does not depend on the format selected in the application settings.



Download link

All versions are here.

Version history

3.5.2

Enhancements:

- Migration to Avalonia UI 12.x.x.

Bug fixes:

- The tooltip in the service window has been translated.
- Modbus: changing the number format in the write field now updates the validation error message.

3.5.1

Enhancements:

- Added support for Korean and Portuguese languages.

Bug fixes:

- When writing multiple flags (function 0x0F), the last byte of data was lost if it was equal to 0.
- Fixed incorrect display of float numbers in Modbus monitoring mode.

3.5.0

Enhancements:

- Added support for multiple interface languages: Russian, English, Hindi, Chinese (Simplified), German, Spanish, French.

3.4.0

Enhancements:

- Modbus: monitoring mode added.
- Modbus scanner: added templates for polling functions.
- Minor fixes and refactorings.

3.3.3

Enhancements:

- Now, when working with macros, only one window is used (working field or editing). The unused window is hidden.
- Refactoring Unit tests.

3.3.2

Enhancements:

- Modbus: now if a timeout is triggered, the contents of the request are displayed.
- The functionality of the "About Application" window has been expanded.

Bug fixes:

- Rare freeze when writing/reading Modbus.
- Continues cyclic polling if an error occurs.
- Incorrect display of the number of Modbus registers written after sending a macro command.
- When sending individual macro commands, a single Slave ID is not taken into account.
- When using a single Slave ID, errors from the Slave ID fields of commands are taken into account in the macro.

3.3.1

Changes:

- The Modbus scanner has added a choice of protocol (Modbus RTU or Modbus ASCII), which is used to search for devices.
- Shortened some names in the UI.

3.3.0

Changes:

- Added the ability to use a single Slave ID for a Modbus macro.
- Now all errors that appear when running a macro are collected into a single message, rather than being shown separately.
- Improvements have been added to make working with the macro window more convenient.
- Fixed a bug with getting an incorrect path when selecting a folder or file.
- Optimizations.
- Refactoring.

3.2.1

Changes:

- MessageBox now has icons that depend on the type of message.
- An “Error Report” has become available in the MessageBox, which appears for messages with an error type. The report can be viewed in a separate window, copied to the clipboard or to a text file.

3.2.0

Changes:

- Work with macros has been expanded. The macro is divided into commands. Now you can send multiple messages at once in one macro.
- DI has been introduced into the project.
- Refactoring. Reduced connectivity between components.
- Minor bugs fixed.

3.1.0

Changes:

- Added support for Modbus RTU over TCP.
- Added support for Modbus ASCII over TCP.
- Added the ability to work with bytes in the "No protocol" mode.
- Added macros.
- Added user manual.
- Bug fixes, minor improvements and refactorings.

3.0.0

Changes:

- The project has been moved from WPF to AvaloniaUI.
- Changed design.
- Added Modbus scanner.
- Modbus: Each recording function has its own design option.
- Modbus: added exchange history maintenance.
- Modbus: added the ability to work with binary data.
- Modbus: added the ability to work with float data.
- Errors in version 2.7.0 have been fixed.

2.7.0

Changes:

- First public version.